

MACHINE LEARNING WITH AI AND PYTHON

DAY 2: TREES, ENSEMBLES, CLUSTERING, AND NEURAL NETWORKS



LEARNING OBJECTIVES

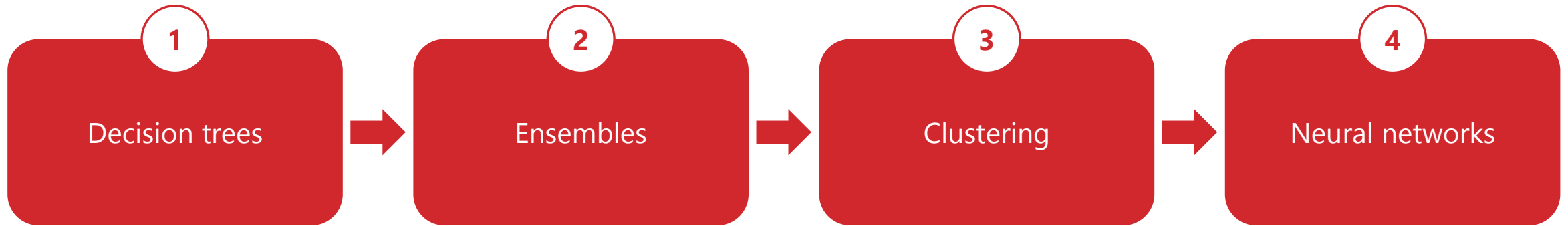
Day 1 gave you the workflow and two models. Today you add four of the most useful tools in machine learning, and the judgment to know when each one fits.

- Build a decision tree, read its rules, and stop it overfitting
- Combine trees into ensembles: random forests and boosting
- Handle imbalanced data so a rare class is not ignored
- Cluster data without labels and judge it with a silhouette score
- Train a first neural network and know when depth is worth it
- Always compare a new model against a simple baseline



YOUR DAY 2 ROADMAP

Four tools, in an order that builds on itself: one tree, then a forest of them, then learning with no answer key, then stacking layers for depth.



Each is a hands-on lab. We teach the idea, then you build it.

DECISION TREES

MODELS YOU CAN READ



FROM LINES TO RULES

Day 1's models drew a boundary with weights. A decision tree instead asks a series of yes/no questions, which produces rules a person can read and check.

Why that matters

- You can print the model and hand it to a domain expert
- It captures non-linear patterns a straight line misses
- Everyday uses: credit approval, medical triage, fault detection

Why this matters:

A tree is the rare model that explains itself, which is why regulated fields still reach for it.

START WITH THE DATA

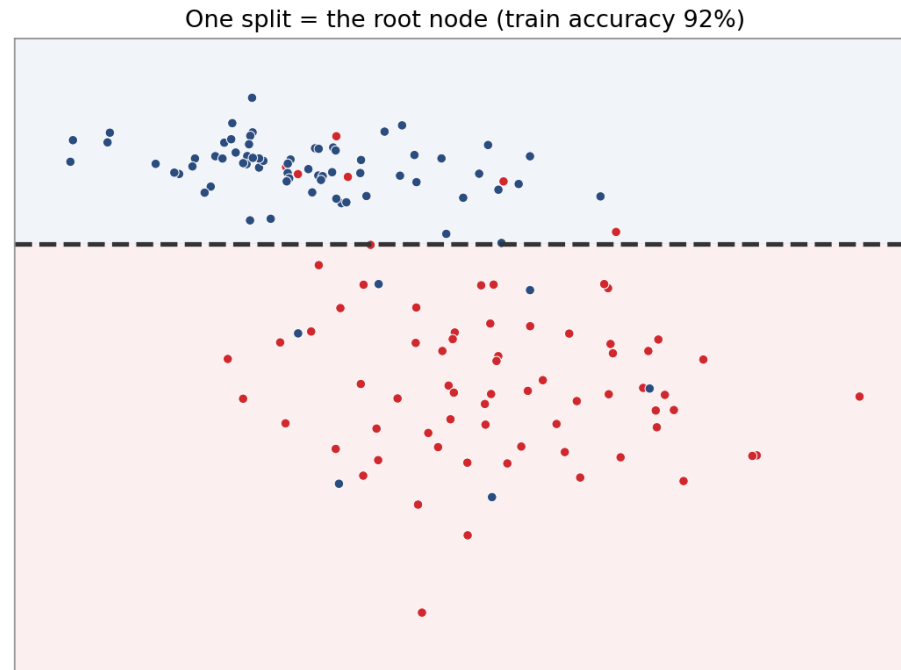
Before any model, look at the data. Here are two classes and two features. A tree's whole job is to carve this plane into regions, one straight cut at a time.



Where would you draw the first line to separate red from blue?

ONE LINE, ONE DECISION

The tree's first move is a single straight cut: pick one feature and one threshold that separate the classes as cleanly as possible. That one cut already gets most points right.



One split, chosen to divide the classes as cleanly as it can: 92% correct from a single line.

A SPLIT IS A QUESTION

That single line is the top of the tree: the root node. A node is just one yes/no question about one feature. Follow the answer down until you reach a leaf, which gives the prediction.

The pieces

- The root node is the first, most important split
- Each branch is the yes/no answer to one question
- Each leaf is the final prediction for whatever lands there

The idea:

So a decision tree is a flowchart the model builds for itself from the data.

HOW A TREE DECIDES: GINI IMPURITY

The tree chooses every split by asking which question best separates the classes. Gini impurity is how it measures 'mixed' versus 'pure'.

Reading Gini

- Gini 0.0 means a pure node: one class only
- Gini 0.5 means perfectly mixed: two classes, 50/50
- The tree picks the split that drops Gini the most

Key point:

In the picture, dark nodes are pure and pale nodes are mixed. Training turns pale into dark, one split at a time.

HOW THE TREE FINDS THE BEST SPLIT

How does the tree pick that first line? It tries many candidate cuts and scores each by how pure the two sides become, then keeps the winner. Purity is measured by Gini impurity.

The search

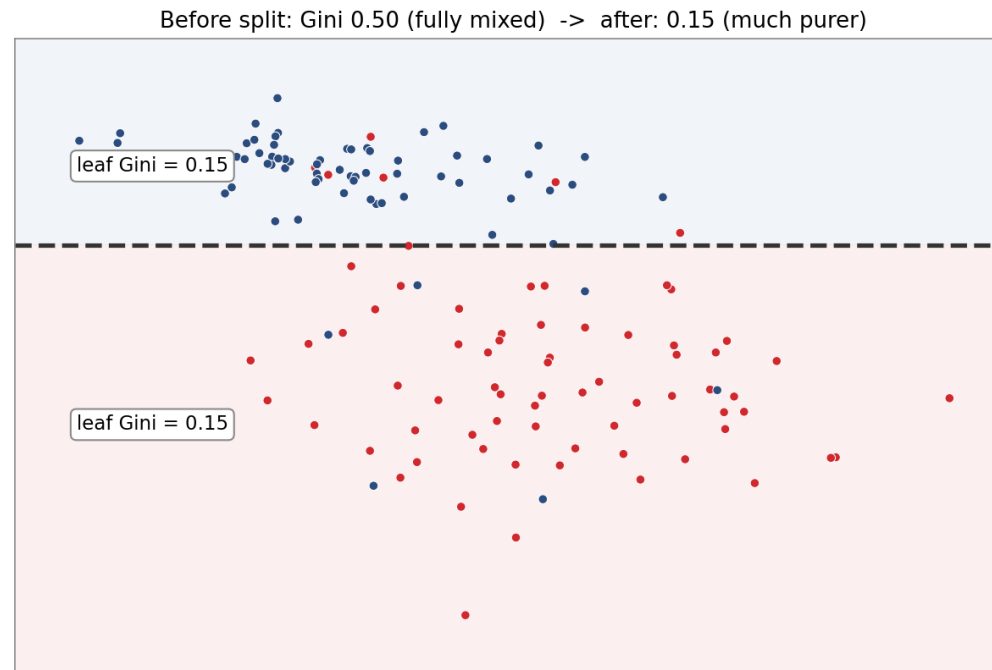
- Try each feature and many thresholds
- Score each cut by the weighted Gini of the two sides
- Keep the cut that lowers impurity the most, then repeat on each side

Key point:

It is greedy: it takes the best cut available right now, then splits each side again.

THE SPLIT THAT PURIFIES MOST

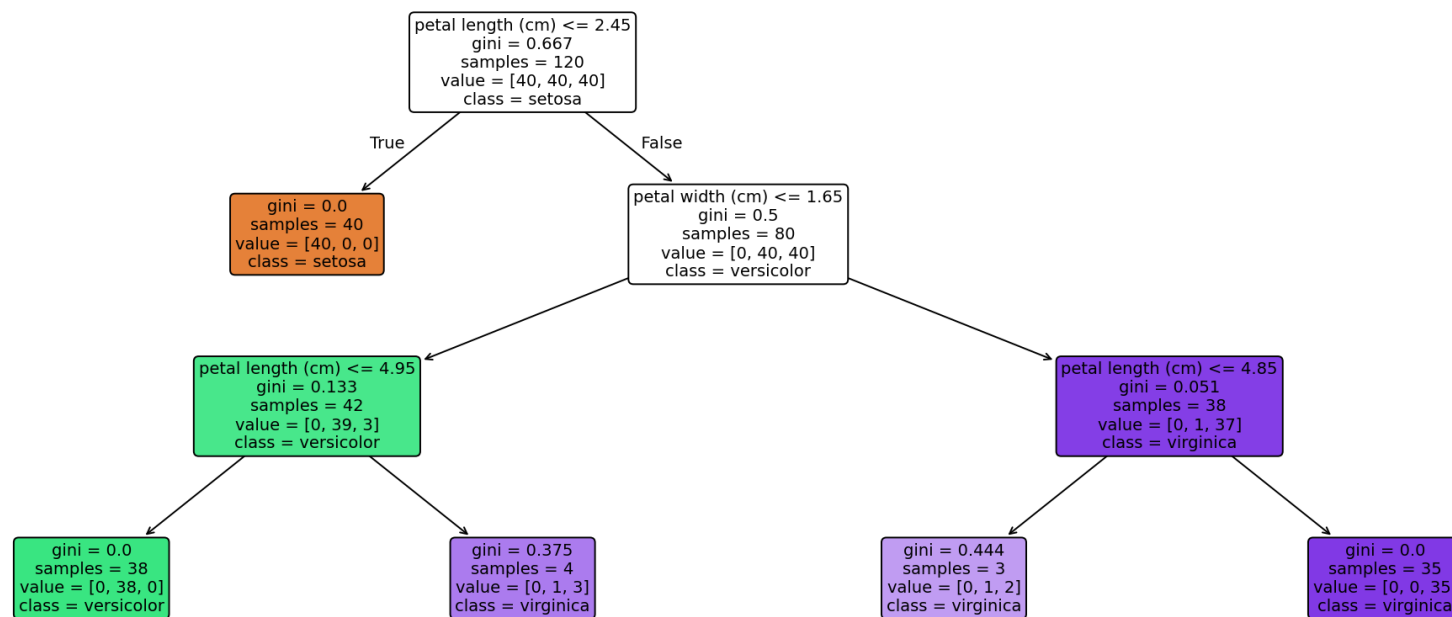
Before the split the node is fully mixed, with Gini near 0.5. The chosen line sends mostly-blue one way and mostly-red the other, so each side is far purer. That drop in Gini is exactly what the tree maximizes.



Parent Gini 0.50 (fully mixed) drops to about 0.15 after the split: much purer.

STACK SPLITS AND YOU CAN READ THE TREE

Each box is one split, just like the line we drew. Stack many of them and the model becomes a flowchart you can read top to bottom, which is why trees are prized where decisions must be explained.



Trace a flower with petal length 1.2 cm from the root: which leaf, which species?

POP QUIZ: SPLITS AND GINI

A tree considers two splits. Split A leaves sides with Gini 0.05 and 0.08; split B leaves 0.30 and 0.28. Which does the tree choose, and why?



2 MINUTES

POP QUIZ: SPLITS AND GINI

A tree considers two splits. Split A leaves sides with Gini 0.05 and 0.08; split B leaves 0.30 and 0.28. Which does the tree choose, and why?

Answer:

Split A. Lower Gini means purer sides, and the tree always keeps the cut that most reduces impurity.

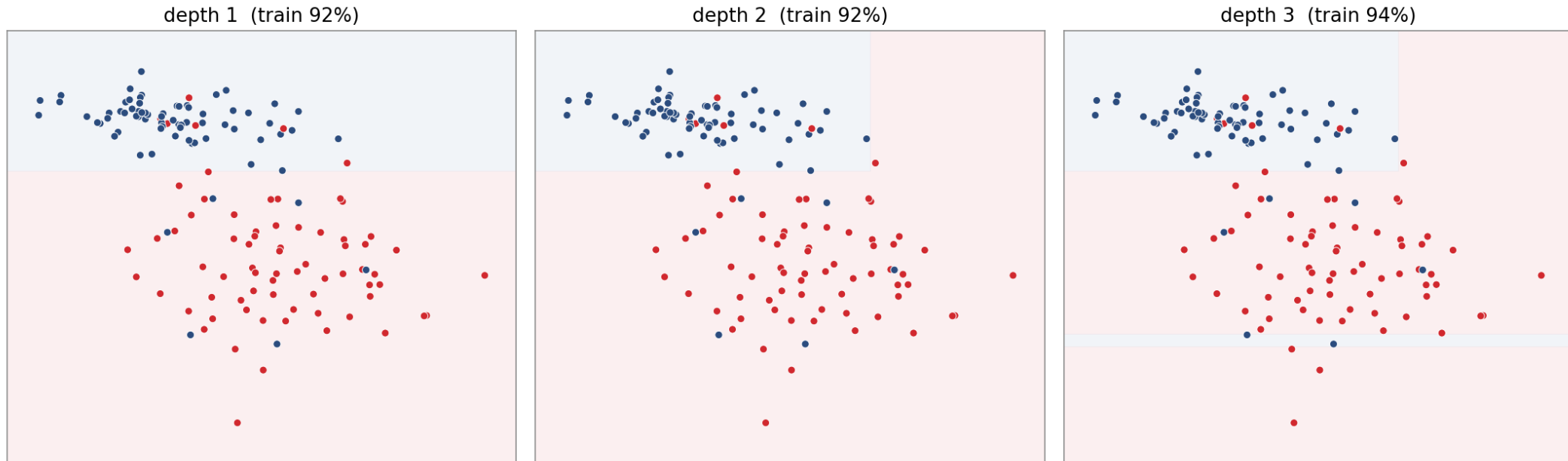
- Gini measures how mixed a node is (0 = pure)
- The tree scores splits by the weighted Gini of both sides
- It greedily keeps the split that drops impurity the most



ADD MORE LINES, FIT MORE

One line is rarely enough. The tree splits each region again and again, and the boundary becomes a finer staircase that fits the training data more and more closely.

More splits carve a finer staircase and fit the training data better



Depth 1 to 3: each new level adds cuts and nudges training accuracy up.

WHY A TREE OVERFITS

If the tree keeps splitting with no limit, it will not stop until every leaf is pure, drawing tiny regions around individual points. At that point it is memorizing the training data, noise and all.

The warning sign

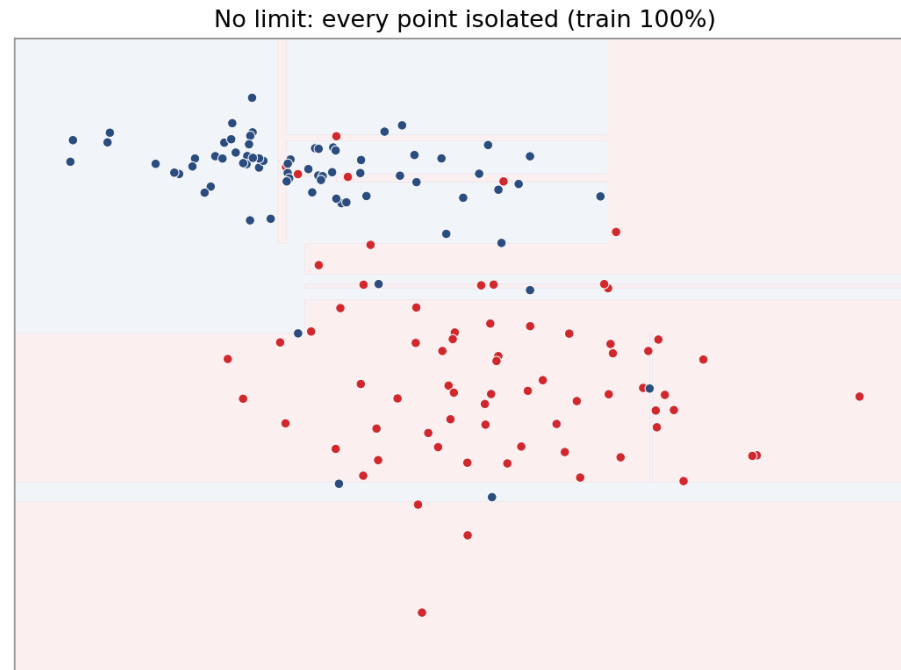
- Unlimited depth drives training accuracy to 100%
- Those tiny regions describe noise, not a real pattern
- A model that memorizes cannot generalize to new data

Watch out:

Perfect on training is a red flag, not a trophy.

PUSH A TREE TOO FAR

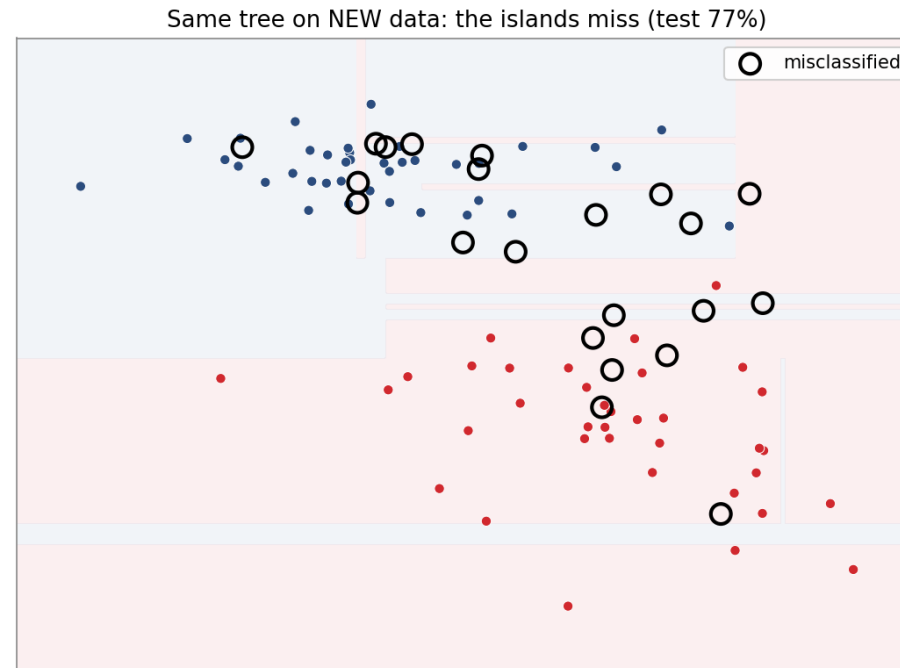
With no depth limit, the tree isolates every last point, including the ones sitting in the other class's territory. The boundary becomes a patchwork of little islands, and training accuracy hits a perfect 100%.



No limit: the tree carves an island around each stray point (train 100%).

THE OVERFIT TREE MEETS NEW DATA

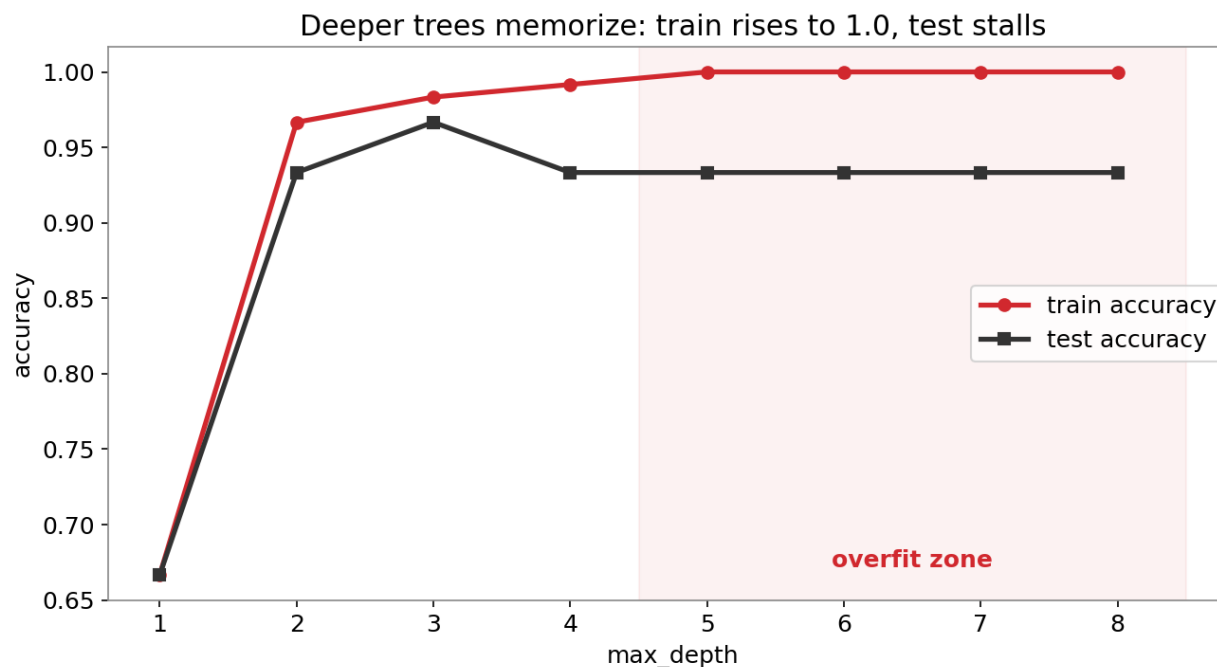
Now take that same memorized tree and show it data it has never seen. The little islands were built around specific training points, so new points fall on the wrong side of them and get misclassified.



Same tree, new data: accuracy drops to 77%. The circled points landed in the wrong island.

OVERFITTING, MEASURED

The boundaries showed overfitting; the numbers confirm it. As depth grows, training accuracy climbs to a perfect 1.0 while test accuracy peaks early and then slips. The gap between the lines is the overfit.



The gap between the two lines is the overfit. Past a point, deeper only memorizes noise.

MAX_DEPTH: THE MAIN LEVER

Shallower trees make coarser splits that generalize better. `max_depth` is the first and most important knob for controlling a tree.

How to tune it

- Try a few depths and watch train and test move together
- Stop where test accuracy peaks, before it dips
- Cross-validation gives a steadier estimate than one split

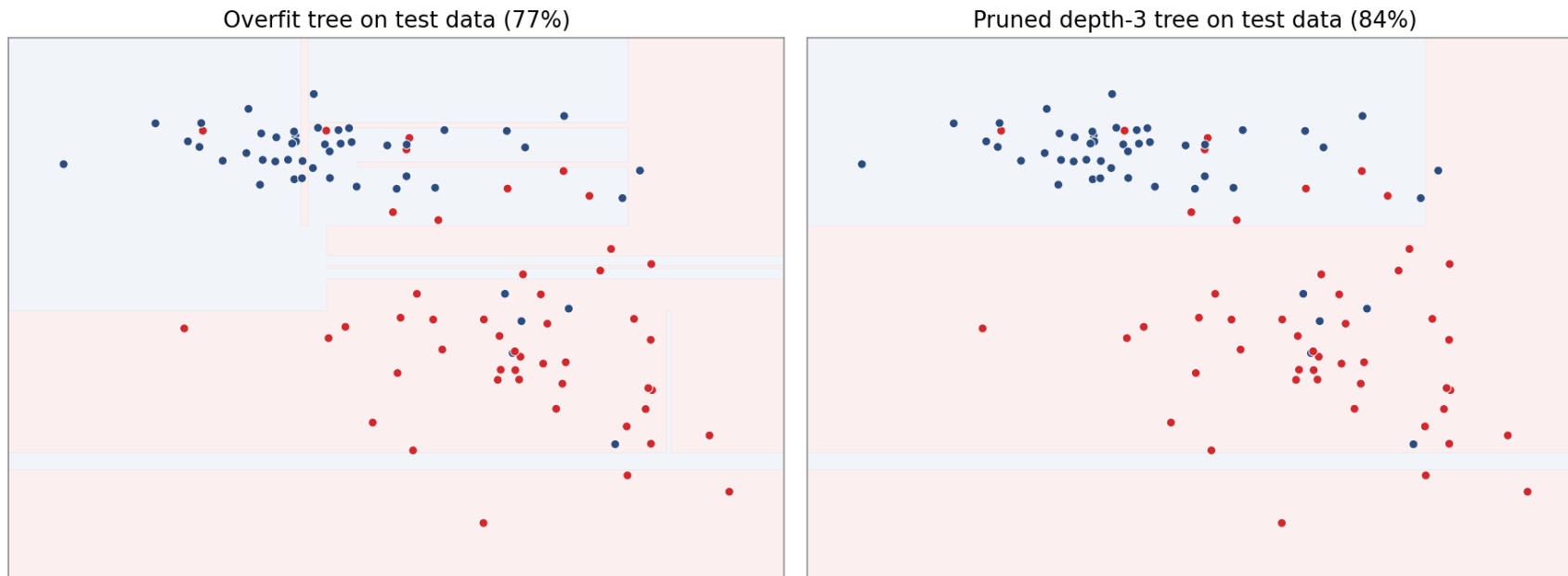
Preview:

Same idea returns for neural networks later, where early stopping plays the role `max_depth` plays here.

PRUNING GENERALIZES BETTER

Limiting the depth prunes away those overfit islands. On the very same unseen data where the full tree scored 77%, a shallow depth-3 tree scores 84%: a simpler model that generalizes better.

A simpler tree generalizes better on data it has never seen



On unseen test data: the pruned depth-3 tree (84%) beats the overfit tree (77%).

FIT AND TUNE A TREE

The same fit and score workflow from Day 1, now with a decision tree. Watch the full tree memorize, then limit the depth.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

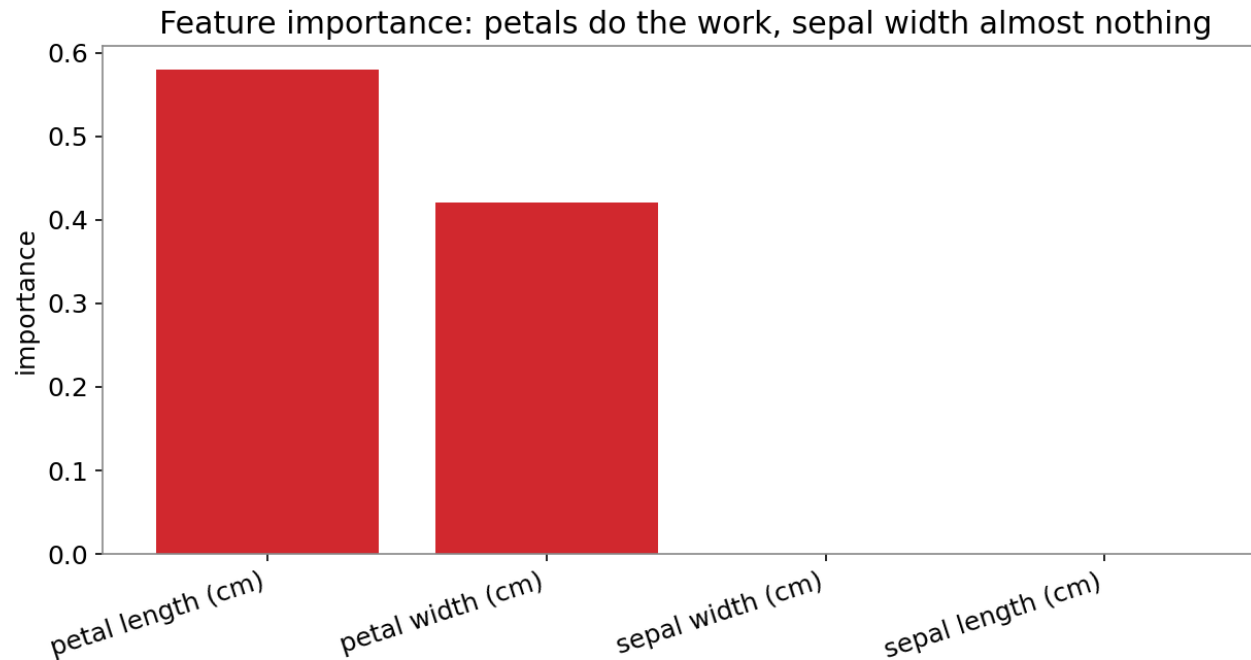
# no limit: the tree memorizes the training data
full = DecisionTreeClassifier(random_state=42).fit(X_tr, y_tr)
print(full.score(X_tr, y_tr), full.score(X_te, y_te)) # 1.0 vs lower

# max_depth is the main lever against overfitting
tree = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_tr, y_tr)
print(round(tree.score(X_te, y_te), 3)) # generalizes better
```

Use case: a shallow tree you can print and hand to a domain expert.

WHAT THE TREE RELIED ON

After fitting, the tree reports how much each feature reduced impurity across all splits. Features it never used score zero.



Petal measurements do the work; sepal width contributes almost nothing, so it is a candidate to drop.

POP QUIZ: OVERFITTING

A decision tree scores 1.00 on the training set but 0.85 on the test set. What happened, and what is the first knob you would turn?



2 MINUTES

POP QUIZ: OVERFITTING

A decision tree scores 1.00 on the training set but 0.85 on the test set. What happened, and what is the first knob you would turn?

Answer:

It overfit: it memorized the training data. Lower max_depth so it makes coarser, more general splits.

- Perfect train with lower test is the overfit signature
- max_depth is the main lever for a decision tree
- Judge on the test set, never on the training score



STRENGTHS AND LIMITS OF TREES

One tree is readable and needs no feature scaling, but it is unstable: a small change in the data can reshape it, and it overfits easily. That weakness is exactly what the next tool fixes.

The trade

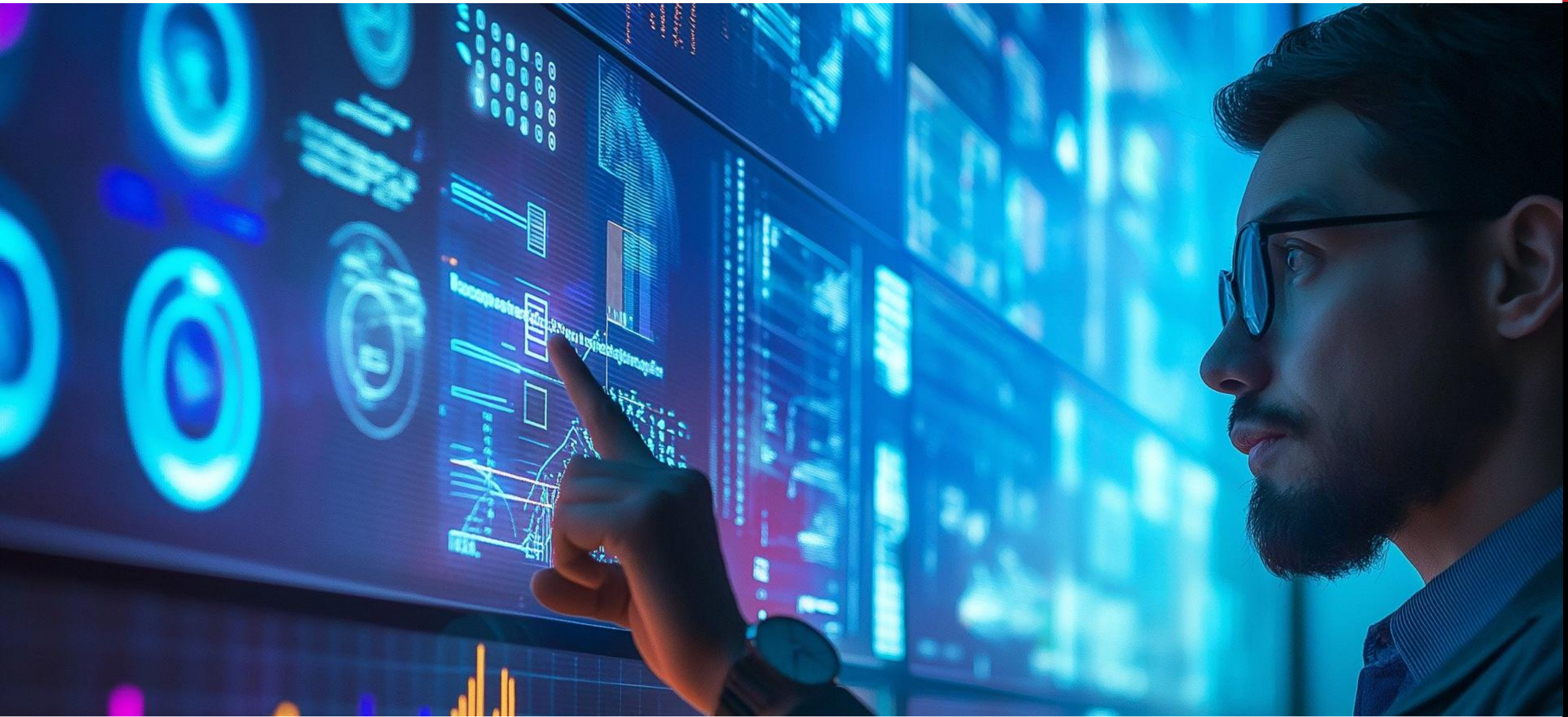
- Strong: interpretable, handles non-linearity, no scaling needed
- Weak: high variance, one tree overfits and swings with the data
- The fix: combine many trees into an ensemble

Bridge:

Hold onto 'high variance'. Averaging many trees is what tames it, which is the whole idea of a random forest.

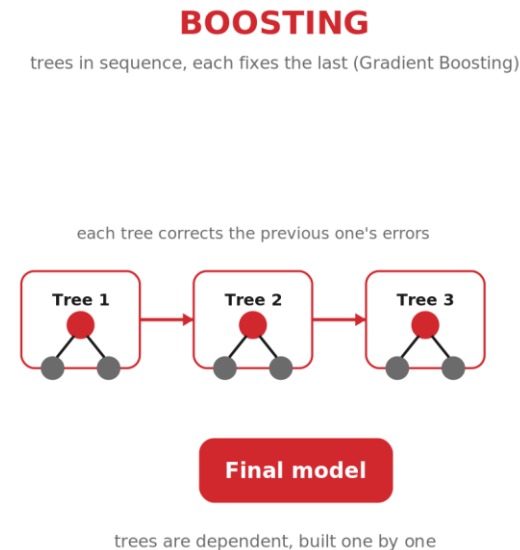
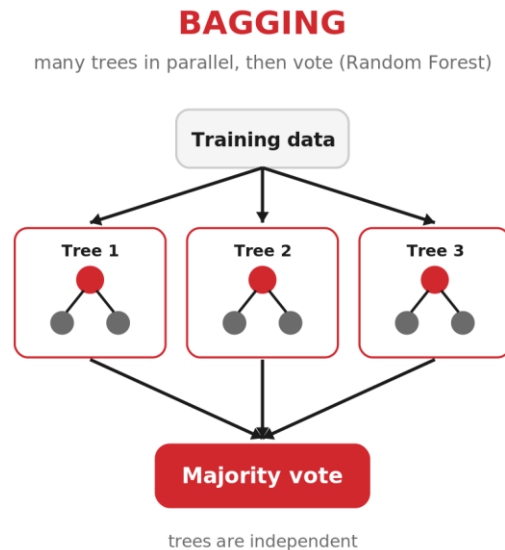
ENSEMBLES AND IMBALANCED DATA

MANY MODELS, HONEST METRICS



BAGGING VS BOOSTING

There are two ways to combine trees. A random forest trains many independent trees and votes (bagging). Boosting trains trees in sequence, each fixing the last one's mistakes.



Bagging reduces variance by averaging; boosting reduces error by focusing on it.

WHY MANY TREES BEAT ONE

A single tree swings with the data. A random forest trains hundreds of trees, each on a random slice of rows and features, then averages their votes. The errors cancel out.

The intuition

- Each tree is a different, imperfect opinion
- Averaging many opinions cancels their random mistakes
- The forest keeps the trees' strengths and loses their instability

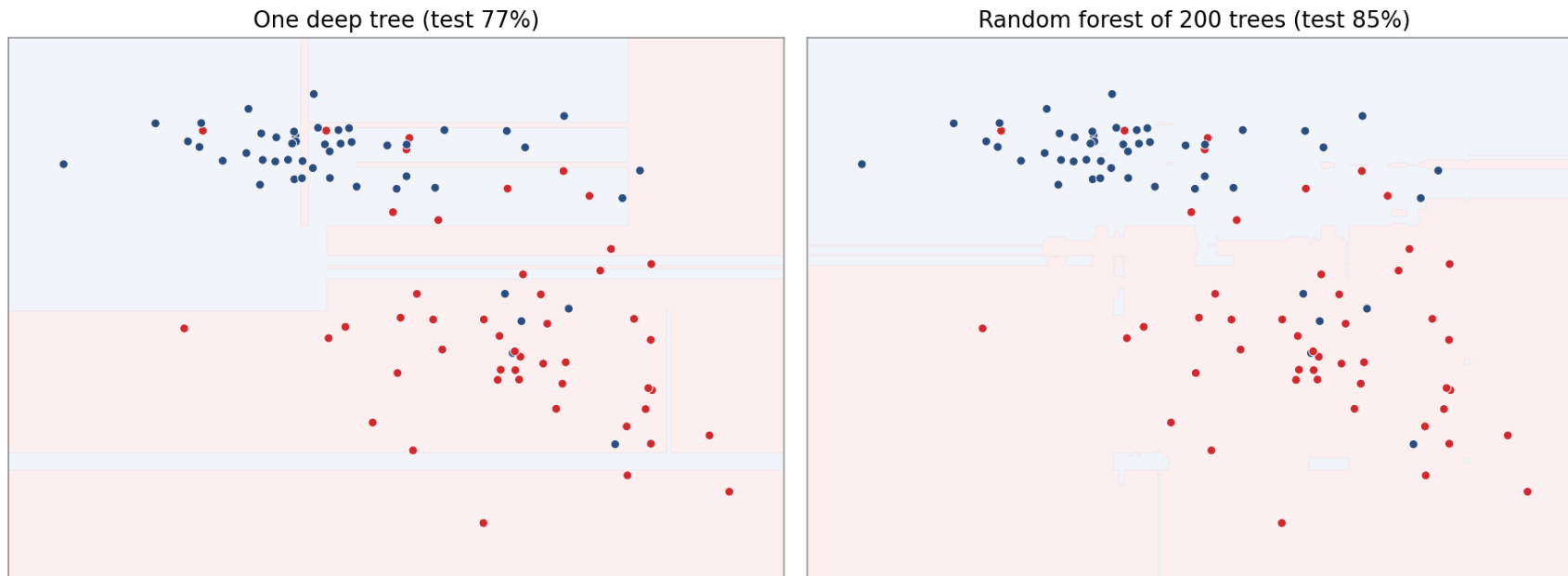
Key point:

This is the wisdom of the crowd, applied to models: many weak, varied voters beat one confident voice.

ONE TREE VERSUS A FOREST

A single deep tree gives that jagged, overfit boundary. Train hundreds of trees on random slices of the data and average their votes, and the boundary smooths out and generalizes better on the same unseen data.

Averaging many trees smooths the jagged boundary and generalizes better



One deep tree (77%) versus a 200-tree random forest (85%) on the same test data.

RANDOM FORESTS IN PRACTICE

A random forest keeps its trees different on purpose, then lets them vote. That built-in variety is what makes the average reliable rather than just repeating one tree's mistakes.

What makes it work

- Each tree trains on a random bootstrap sample of the rows
- Each split considers only a random subset of the features
- More trees rarely hurt accuracy; they mainly cost time

In practice:

Sensible defaults go a long way: a few hundred trees and you are usually near best.

HOW BOOSTING LEARNS FROM MISTAKES

Boosting takes the opposite approach to a forest. Instead of independent trees voting, it builds trees in sequence, each one focused on the examples the previous trees got wrong.

The sequence

- Fit a small tree, then measure where it erred
- The next tree concentrates on those hard cases
- Add the trees up, one careful step at a time

Trade-off:

Powerful on tabular data, but its extra knobs (depth, learning rate) can overfit if pushed too far.

THE TRAP: ONLY 12% CHURN

You need to flag customers likely to cancel. Only 12% churn. A model that predicts 'retained' for everyone scores 88% accuracy while catching zero churners.

What goes wrong

- Accuracy rewards the majority class you do not care about
- The retention team acting on it would be flying blind
- You met this on Day 1; now it bites an ensemble too

Watch out:

High accuracy on imbalanced data is the most common way a model looks good and is useless.

BASLINE RANDOM FOREST

Fit a strong default model first, then look past accuracy to the metric that matters when a class is rare.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, recall_score, f1_score

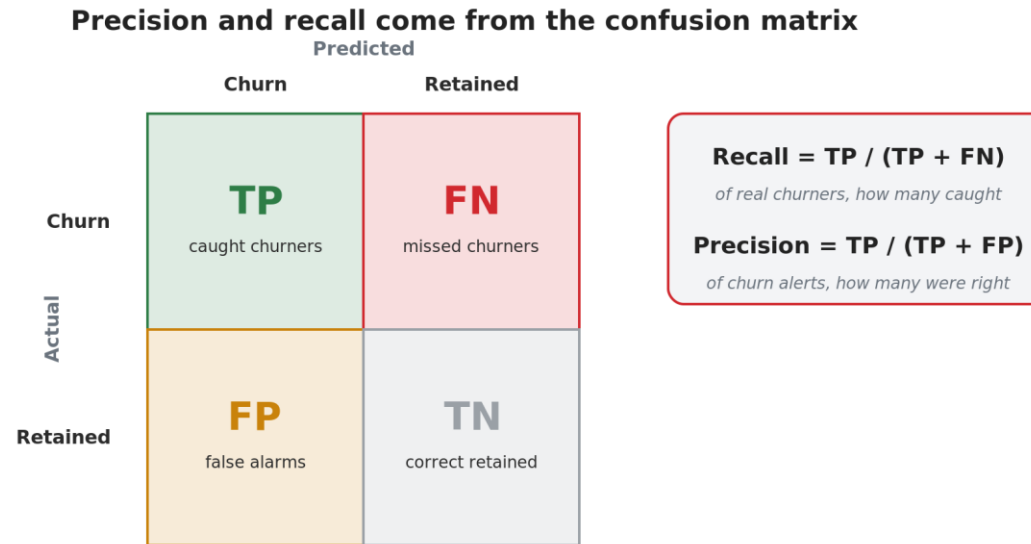
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_tr, y_tr)
pred = rf.predict(X_te)

print('accuracy:', round(accuracy_score(y_te, pred), 3)) # looks great
print('recall   :', round(recall_score(y_te, pred), 3))   # the truth
print('F1       :', round(f1_score(y_te, pred), 3))
```

Accuracy looks strong, but recall on churned customers is low. Judge on recall and F1.

PRECISION VERSUS RECALL

When one class is rare, a single accuracy number hides everything. The confusion matrix splits mistakes into two kinds, and two metrics tell you which kind you are making.



On rare-event problems like churn, recall is usually what you optimize

Recall = of real churners, how many you caught. Precision = of your alerts, how many were right.

WHICH ERROR COSTS MORE?

Precision and recall pull against each other, so you choose which to favor based on which mistake is more expensive in your problem.

Two questions

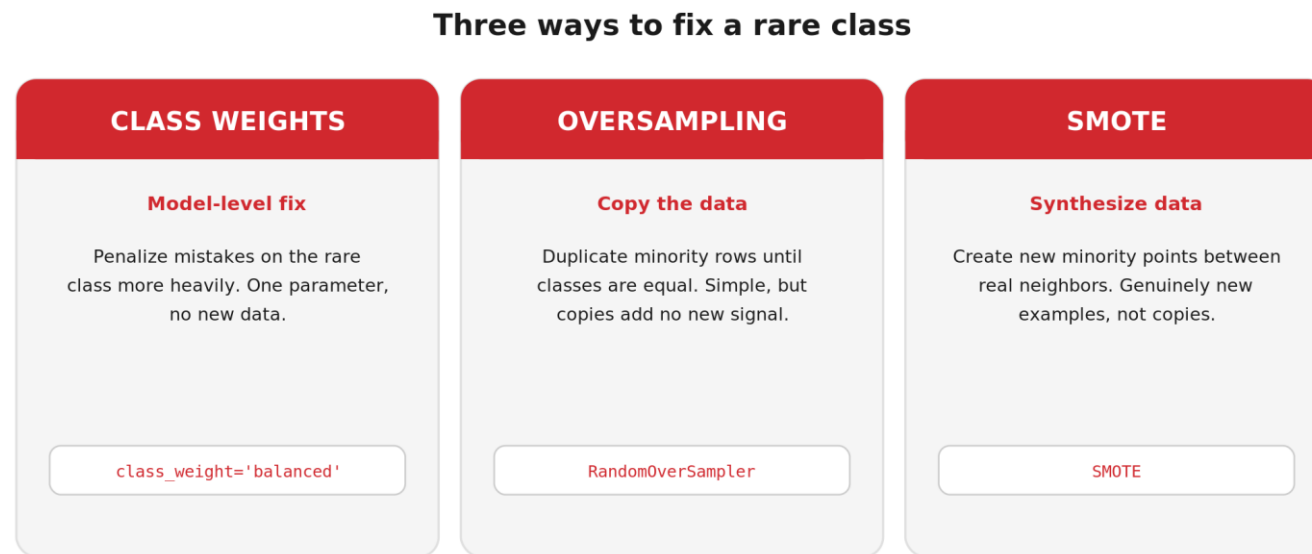
- Missing a churner (a false negative) quietly loses a customer
- A false alarm (a false positive) wastes a retention offer
- You catch more churners by raising recall, at the cost of more false alarms

Rule of thumb:

For churn, fraud, or disease screening, a miss usually hurts most, so recall wins.

THREE WAYS TO FIX IMBALANCE

You can fix the model, or fix the data. Class weights change what the model cares about; oversampling and SMOTE change the training set it learns from.

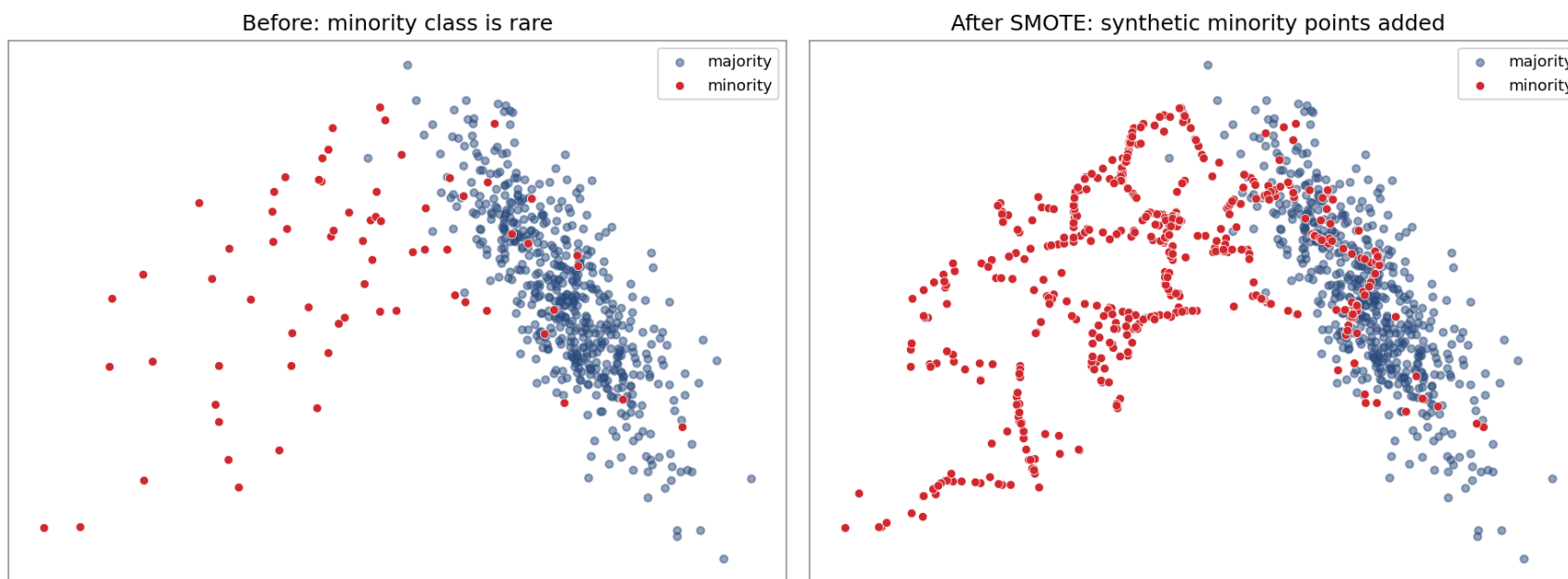


Cheapest first: try class weights, then resampling if you need more.

WHAT SMOTE ACTUALLY DOES

Plain oversampling just copies minority rows. SMOTE is smarter: it invents new, plausible minority examples along the lines between real neighbors, filling in the region instead of stacking duplicates.

SMOTE invents new minority points between real neighbors (not copies)



Left: the minority class is rare. Right: SMOTE adds synthetic points between real neighbors.

REBALANCE THE TRAINING DATA

Class weights are a parameter; oversampling and SMOTE reshape the training set. Whatever you choose, the test set stays untouched.

```
from imblearn.over_sampling import RandomOverSampler, SMOTE

# 1) model-level: penalize minority errors more
RandomForestClassifier(class_weight='balanced', random_state=42)

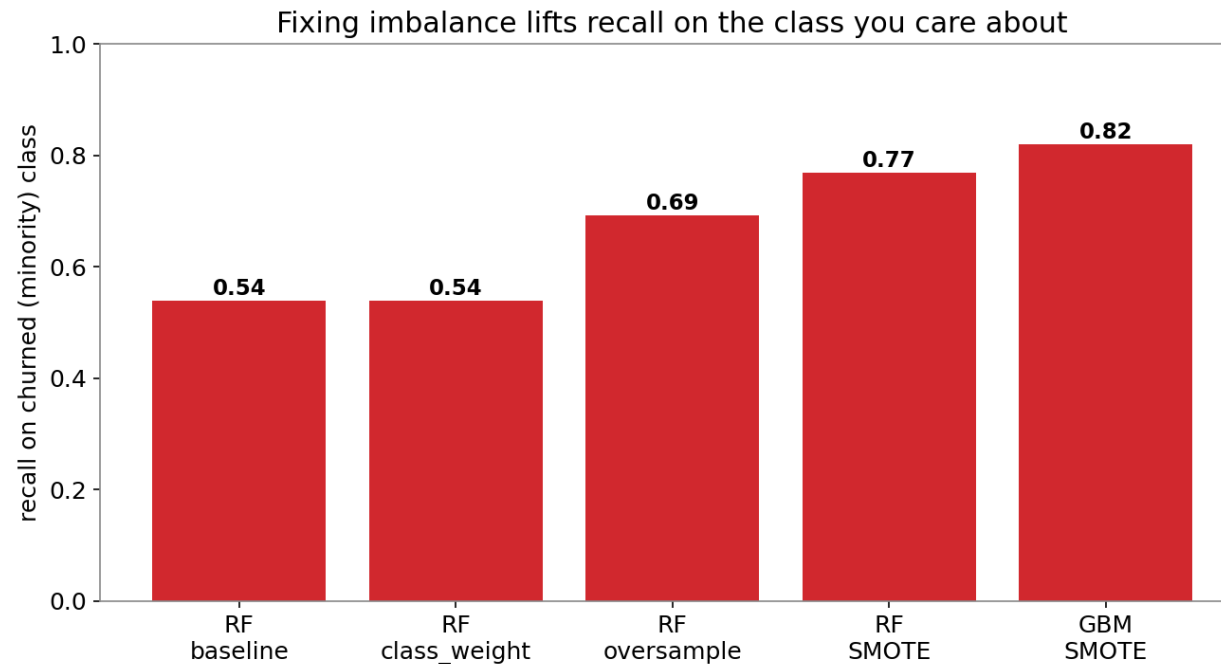
# 2) duplicate minority rows until the classes are equal
X_ros, y_ros = RandomOverSampler(random_state=42).fit_resample(X_tr, y_tr)

# 3) synthesize new minority points between real neighbors
X_sm, y_sm = SMOTE(random_state=42).fit_resample(X_tr, y_tr)
# then fit a fresh model on the balanced training data
```

Resample the training set only. Always evaluate on real, untouched data.

WHAT EACH FIX BUYS YOU

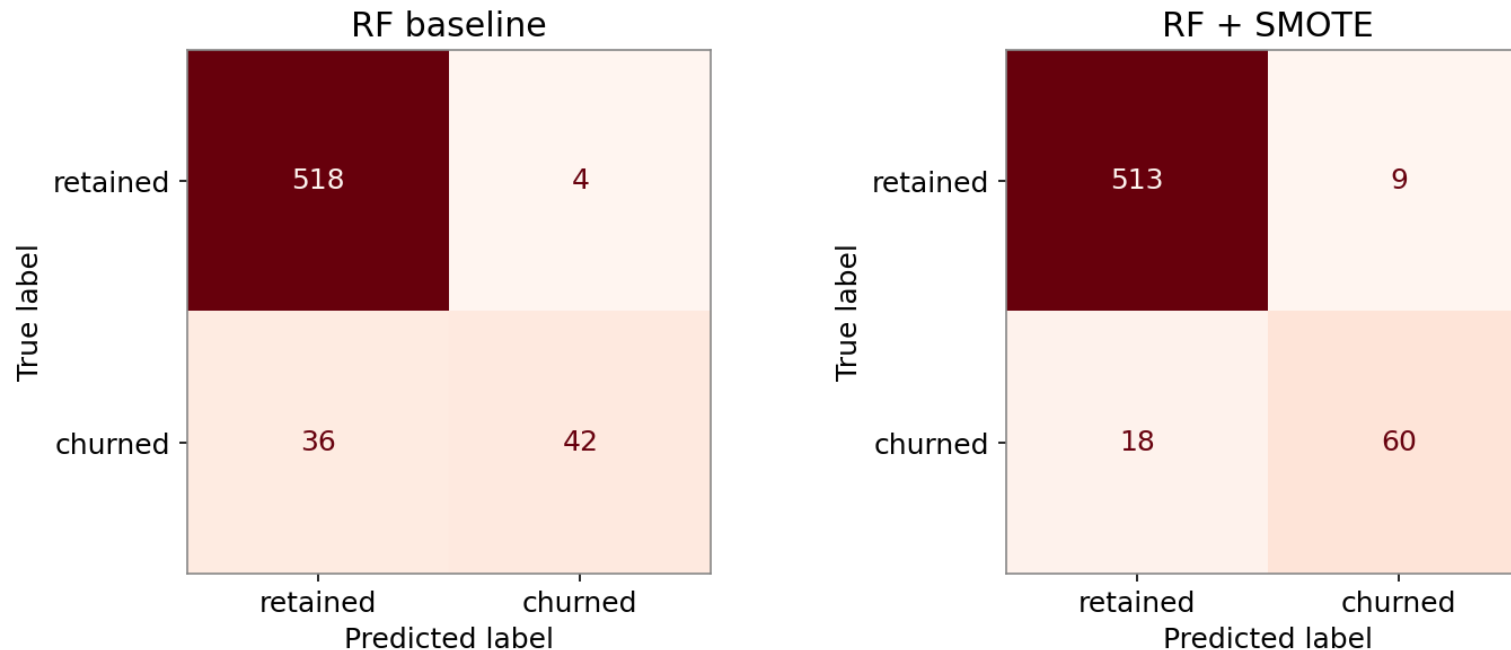
Measured on the same held-out data, each fix lifts recall on the churned class. More informative resampling tends to help more.



Class weights barely moved here; SMOTE and boosting recovered far more churners.

WHERE THE ERRORS MOVED

The summary numbers hide where mistakes land. Compare the baseline and the fixed model side by side to see exactly what you traded.



Missed churners (bottom-left) dropped from 36 to 18. The cost was more false alarms.

BAGGING VS BOOSTING: WHEN TO USE

Both combine trees, but they behave differently. Knowing which to reach for saves you tuning time.

Rules of thumb

- Random forest: a robust default, hard to break, little tuning
- Gradient boosting: often best on tabular data, but tune it carefully
- Start with a forest; try boosting when you need the last few points

Watch out:

More power raises the stakes: boosting can overfit if you push depth and learning rate too far.

POP QUIZ: THE ACCURACY TRAP

Only 12% of customers churn. A model predicts 'retained' for everyone and reports 88% accuracy. Why is that useless, and what do you measure instead?



3 MINUTES

POP QUIZ: THE ACCURACY TRAP

Only 12% of customers churn. A model predicts 'retained' for everyone and reports 88% accuracy. Why is that useless, and what do you measure instead?

Answer:

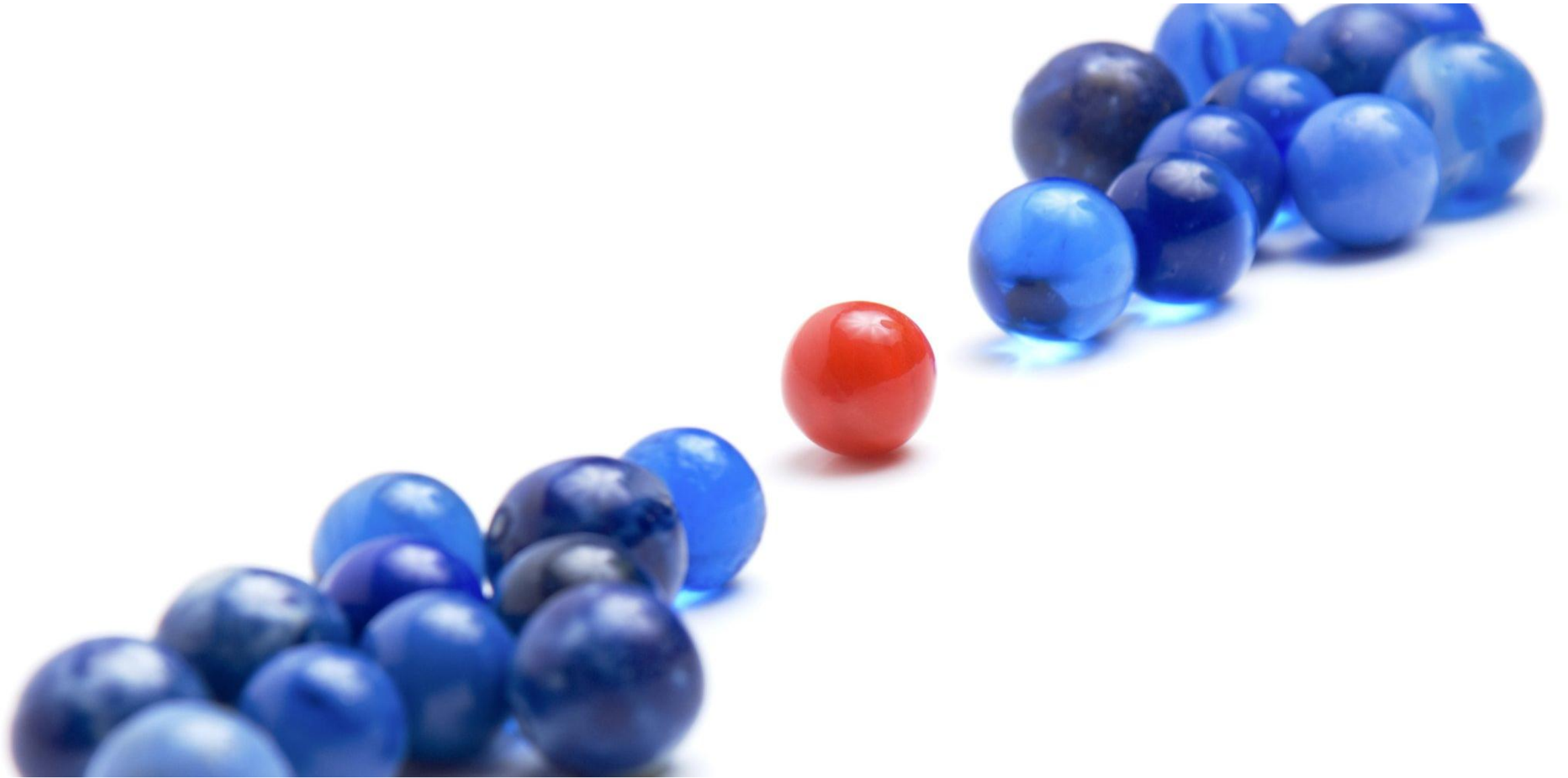
It never catches a churner, so it is useless despite 88% accuracy. Measure recall, and F1, on the churned class.

- Accuracy rewards the majority class you do not care about
- Recall asks: of the real churners, how many did we catch?
- Fixes: class_weight, oversampling, or SMOTE, then re-measure



CLUSTERING

FINDING STRUCTURE WITHOUT LABELS



LEARNING WITHOUT LABELS

Every model so far had an answer key: a price, a species, a churn flag. Clustering removes that. The goal is to discover structure that is already in the data.

Where it fits

- Customer segments, document grouping, anomaly detection
- No labels to train on, and no accuracy to optimize
- You judge results by whether the groups are useful

Why this matters:

The hard part is not the algorithm, it is evaluating a model when you have nothing to check it against.

K-MEANS IN THREE STEPS

K-means is the simplest clustering method and the right starting point. It repeats a simple loop until the groups settle.

The loop

- Place k centers, assign each point to its nearest center
- Move each center to the middle of its assigned points
- Repeat until assignments stop changing

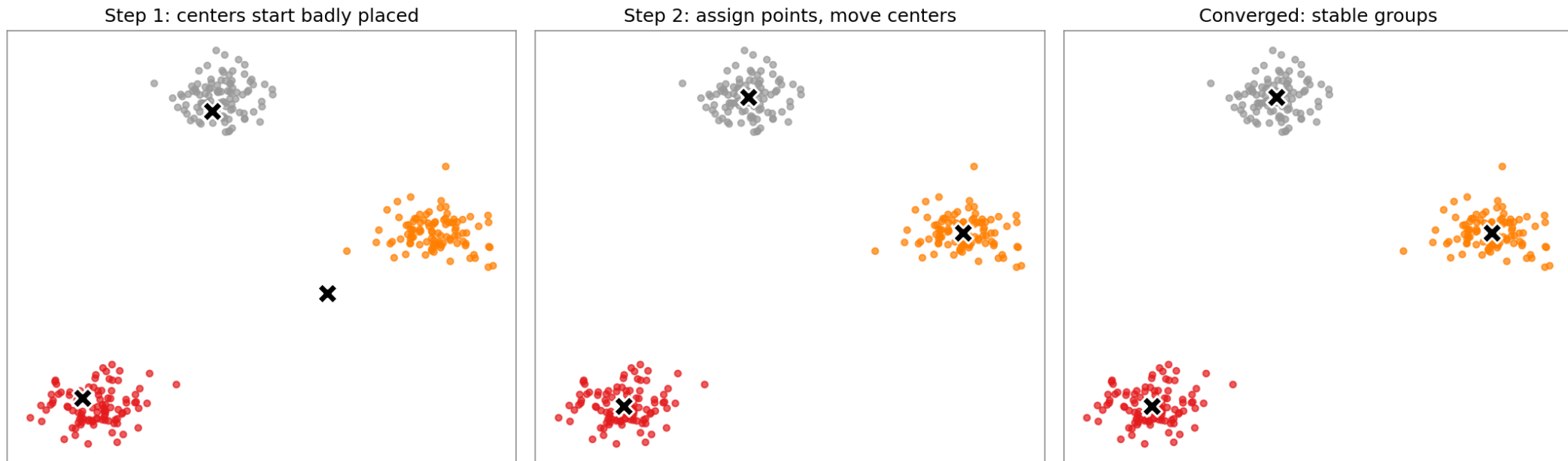
Key point:

You choose k before fitting. The elbow method, next, helps you pick it.

WATCH K-MEANS CONVERGE

K-means often starts with its centers in the wrong place, then repeats one simple loop: assign each point to its nearest center, move each center to the middle of its points, and repeat. In a few passes it settles.

K-means repeats one loop: assign points, move centers, repeat until stable



A bad start, then assign-and-move repeats until the centers stop moving.

WHAT K-MEANS ASSUMES

K-means is fast and simple, but it makes assumptions. Knowing them tells you when to trust it and when to reach for something else.

The fine print

- You must choose k before it runs
- It looks for round, similar-sized blobs
- A poor random start can give a poor result, so it runs several times

Good to know:

When clusters are stretched or oddly shaped, k-means struggles and other methods handle it better.

SCALE FIRST (WHY TREES DID NOT NEED TO)

K-means measures distance between points. If one feature is on a bigger scale, it dominates every distance. Scaling puts features on equal footing.

The contrast

- Distance-based methods (k-means, k-NN): scaling is required
- Decision trees split one feature at a time: scale does not matter
- StandardScaler centers each feature to mean 0, unit variance

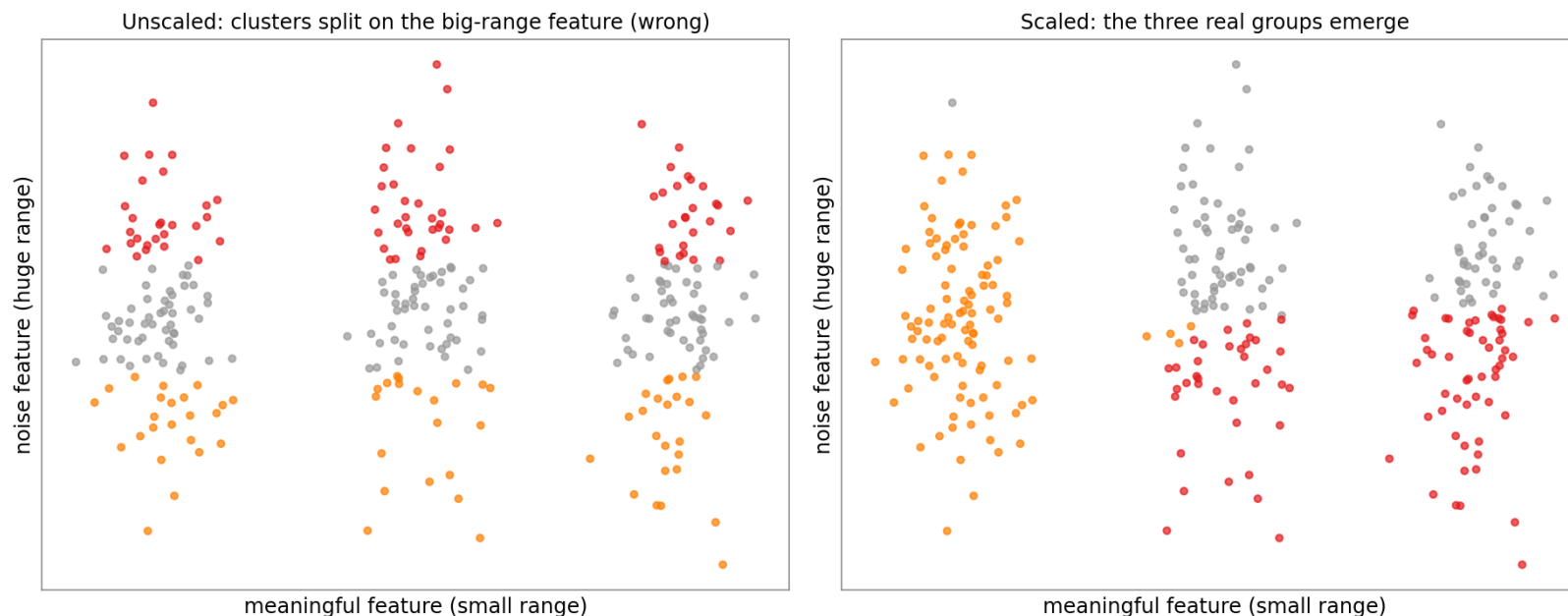
Common question:

A student asked in Lab 1 why we did not scale for the tree. This is the answer: trees are scale-free, distances are not.

WHY SCALING MATTERS

K-means judges everything by distance, so a feature with a huge range dominates every distance and drags the clusters onto that one axis. Scaling first puts every feature on equal footing.

Distance-based methods need scaling, or one feature drowns out the rest



Unscaled, the big-range feature decides everything (wrong). Scaled, the real groups appear.

CLUSTER AND SCORE

Scale, cluster, and then judge quality with a score that needs no labels at all.

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X_scaled = StandardScaler().fit_transform(X)    # distance-based: scale first

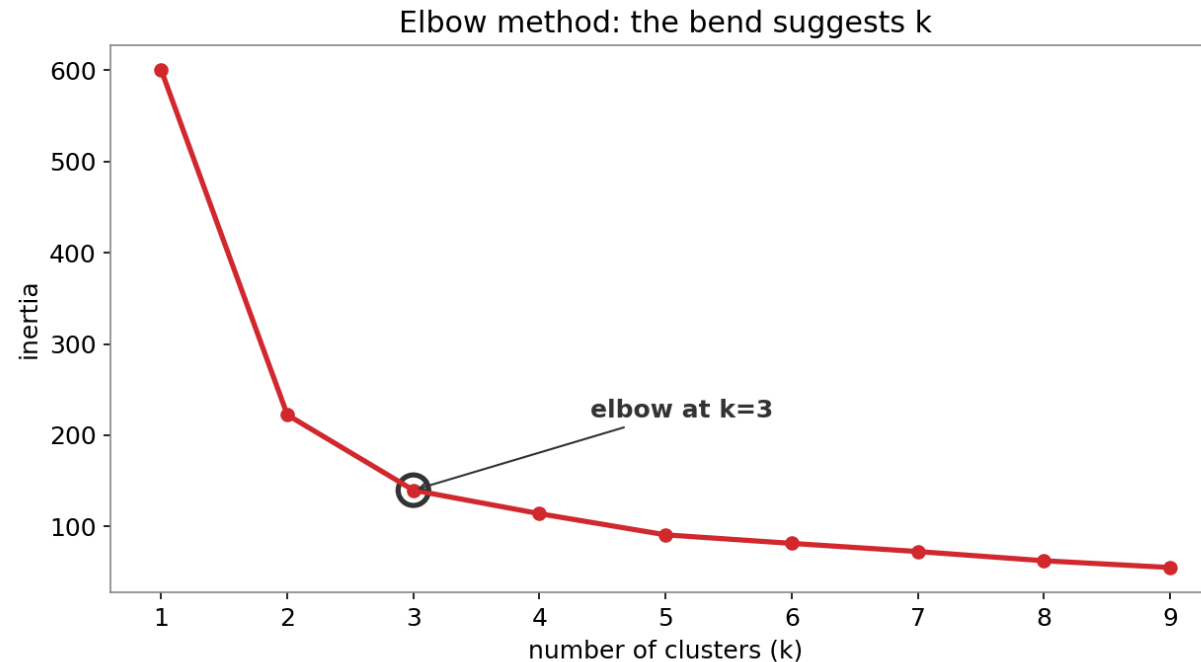
km = KMeans(n_clusters=3, random_state=42, n_init=10).fit(X_scaled)
clusters = km.labels_

print(round(silhouette_score(X_scaled, clusters), 3))    # ~0.55, no labels
```

No answer key: the silhouette score judges cohesion versus separation.

CHOOSING K: THE ELBOW

K-means needs k up front. Fit it for a range of k and plot the inertia. The rate of improvement slows at the natural number of groups, which looks like an elbow.

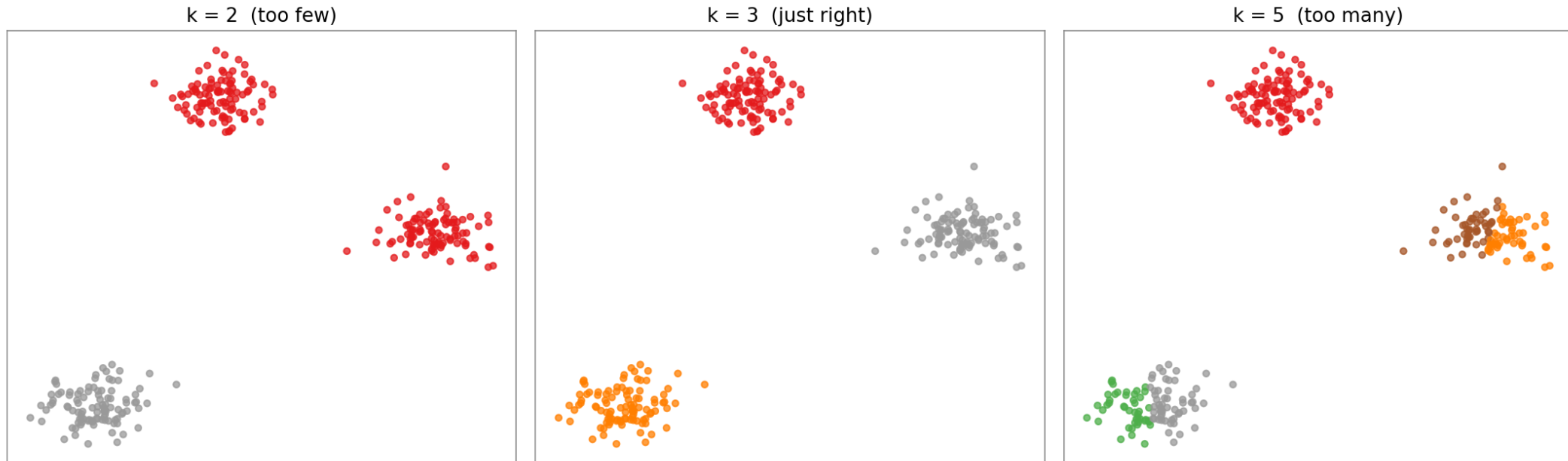


The bend at $k=3$ suggests three groups. It is a heuristic, so confirm it with the silhouette.

TOO FEW, JUST RIGHT, TOO MANY

Because you choose k yourself, it helps to see what wrong choices look like. Too small merges real groups together; too large splits a single group into pieces.

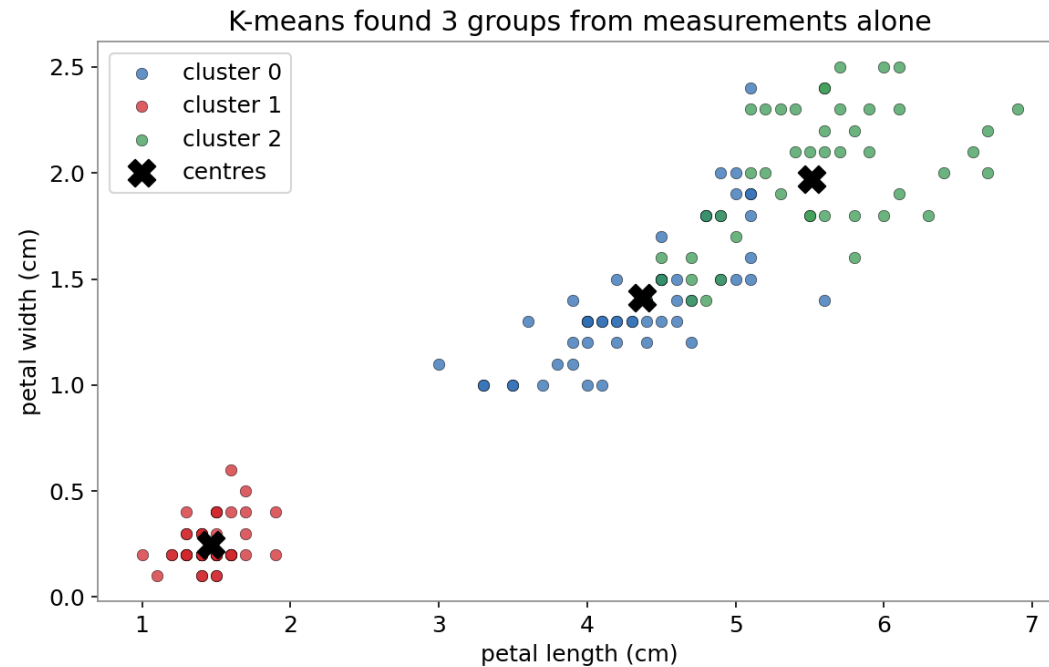
Choosing k : too few merges real groups, too many splits them



$k=2$ merges two real groups; $k=3$ matches the data; $k=5$ splits groups that belong together.

STRUCTURE FOUND FROM NUMBERS ALONE

With k chosen, k -means groups the flowers using only their measurements. It never saw a species label.



One group is cleanly separate; two overlap at the boundary, which the reveal will explain.

EVALUATE WITHOUT AN ANSWER KEY

With no labels, you cannot compute accuracy. The silhouette score fills the gap by measuring how tight and well-separated the clusters are.

Reading the silhouette

- Near 1.0: tight clusters, far apart, well-formed
- Near 0: clusters overlap and blur together
- Below 0: points likely assigned to the wrong cluster

Key point:

Around 0.55 on this data is a solid result. Compare it across k values to confirm your choice.

THE REVEAL: GIVING CLUSTERS MEANING

Because this dataset does have hidden labels, we can grade the clustering, a luxury real projects rarely get. Then we make the numbered clusters mean something.

Two moves

- Crosstab clusters against true species: one is perfect, two overlap
- Read each cluster's feature means to name it
- 'Cluster 2' becomes 'the small-petal group' a stakeholder understands

The point:

K-means recovered the species from measurements alone, which is the payoff of the whole lab.

WHERE CLUSTERING EARNS ITS KEEP

Clustering shines when you have no labels but suspect there is structure, and you want the data itself to suggest groups you did not define in advance.

Real uses

- Customer segments for targeted messaging
- Grouping documents or images by similarity
- Flagging anomalies as points that fit no cluster well

In practice:

The output is a starting point for a human, not a final verdict, so always sanity-check the groups.

POP QUIZ: JUDGING CLUSTERS

You cluster customers and there is no label to check against. How do you tell whether the clustering is any good, and how do you choose the number of clusters?



3 MINUTES

POP QUIZ: JUDGING CLUSTERS

You cluster customers and there is no label to check against. How do you tell whether the clustering is any good, and how do you choose the number of clusters?

Answer:

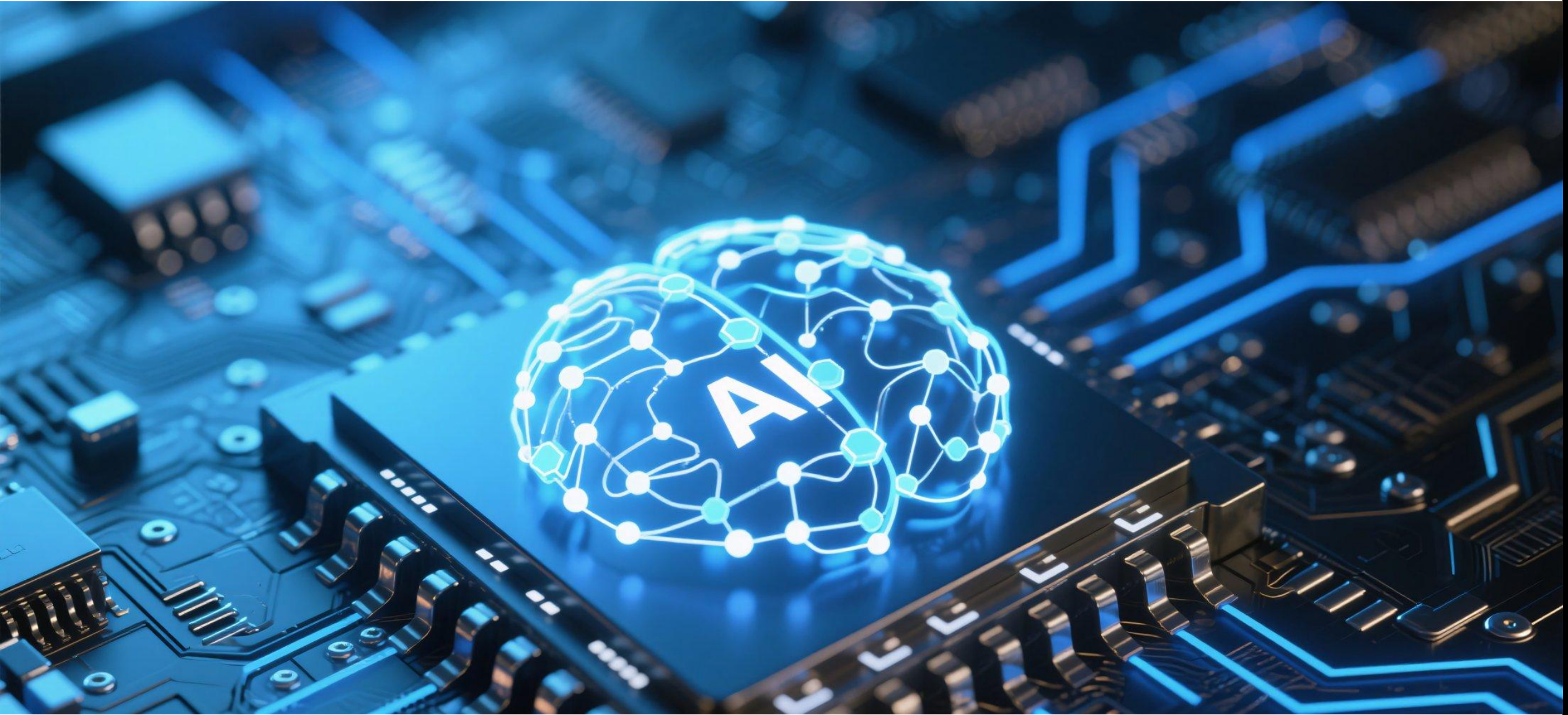
Use the silhouette score to judge quality, and the elbow method to choose k . There is no accuracy to compute.

- No labels means no accuracy or recall
- Silhouette near 1 = tight and separated; near 0 = overlapping
- Confirm k with the elbow, then read per-cluster means for meaning



NEURAL NETWORKS

STACKING LAYERS FOR DEPTH



WHY GO DEEPER

Every model so far made its prediction with a single transformation of the inputs. A neural network stacks many transformations, each learning a higher-level representation of the data.

What depth buys

- Early layers learn simple features, later layers combine them
- This finds patterns flat models miss, especially in images and text
- The cost: slower training, more overfitting, less interpretability

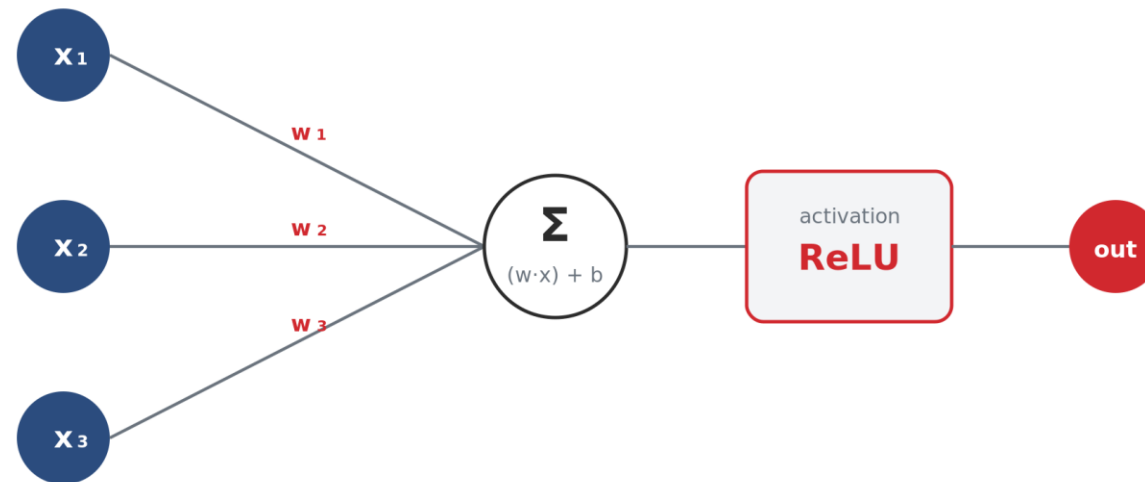
Framing:

Depth is powerful and not free. The rest of this section is the power, the pitfall, and the fix.

INSIDE A SINGLE NEURON

Everything in a neural network is built from this one small unit. A neuron multiplies each input by a weight, adds them up with a bias, and passes the result through an activation function.

One neuron: weigh the inputs, add them up, pass through an activation



A network is just many of these, wired in layers

Weigh the inputs, sum them, then bend the result through an activation like ReLU.

FROM ONE NEURON TO A NETWORK

Stack many neurons into a layer, then stack layers, and you have a network. Each layer transforms the output of the one before it into something more useful.

The structure

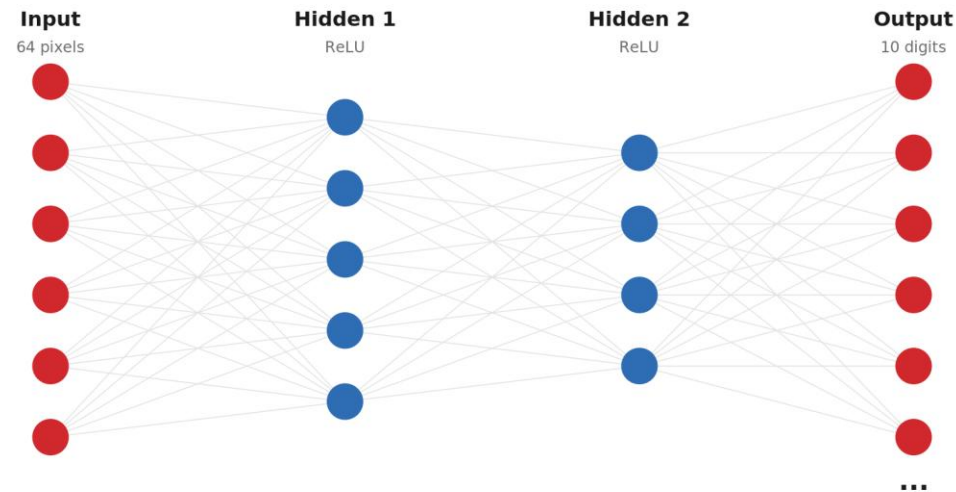
- An input layer, one neuron per feature
- One or more hidden layers that learn patterns
- An output layer, one neuron per class

The idea:

Depth means more layers, and more layers can learn more abstract patterns.

INSIDE A NEURAL NETWORK

A network is layers of neurons. Each connection has a learned weight; each neuron adds its inputs and passes them through an activation. ReLU adds the non-linearity that makes stacking layers worthwhile.



each connection has a learned weight; ReLU adds the non-linearity that lets layers stack

Input (one neuron per pixel), one or more hidden layers that learn features, output (one per class).

WHAT THE HIDDEN LAYERS LEARN

The power of depth is that each layer builds on the last. Early layers pick up simple pieces, and later layers combine them into the concepts you actually care about.

Building up

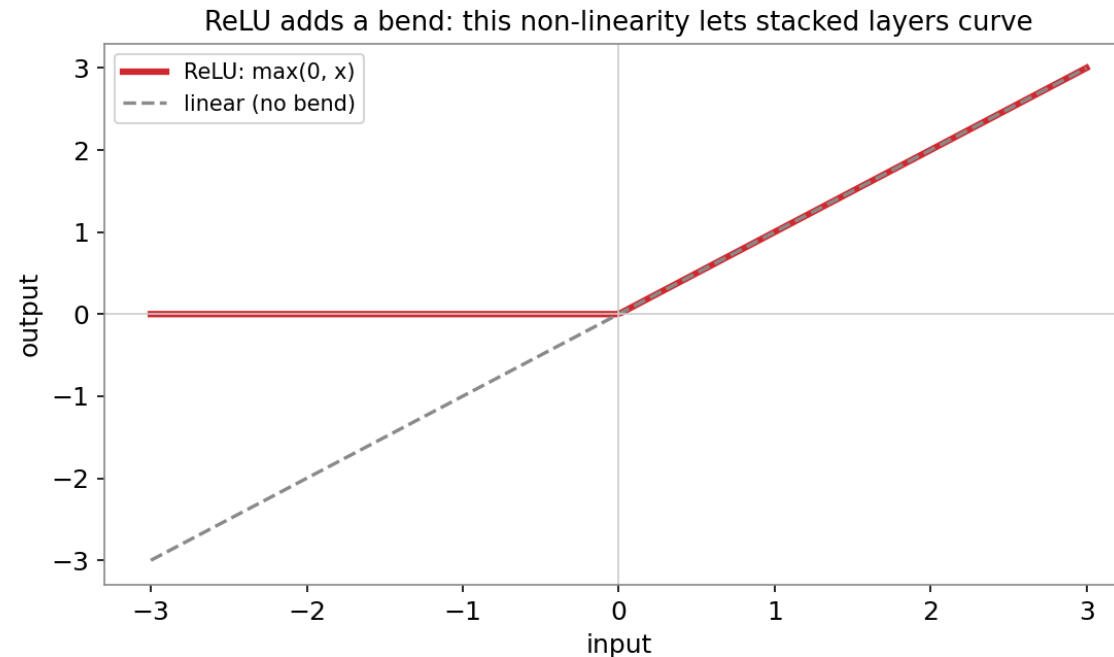
- Early layers detect edges and simple shapes
- Middle layers combine those into parts
- Later layers assemble parts into whole objects

Why it matters:

You do not design these features; the network discovers them from the data.

WHY NON-LINEARITY: RELU

Without an activation, stacking layers is pointless: a stack of straight lines is still just a straight line. ReLU adds a simple bend, and those bends are what let a network fit curved patterns.



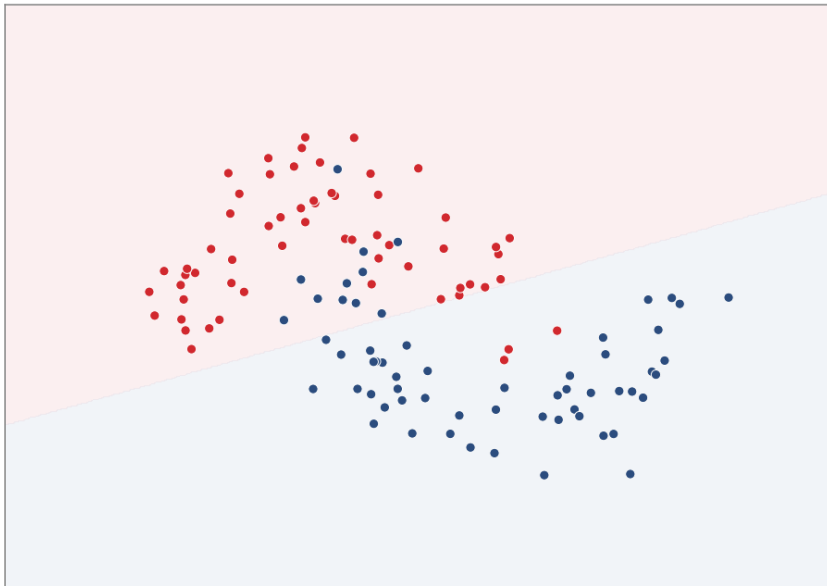
ReLU keeps positive values and zeros out negatives. Small bends stack into complex curves.

WHEN A LINE IS NOT ENOUGH

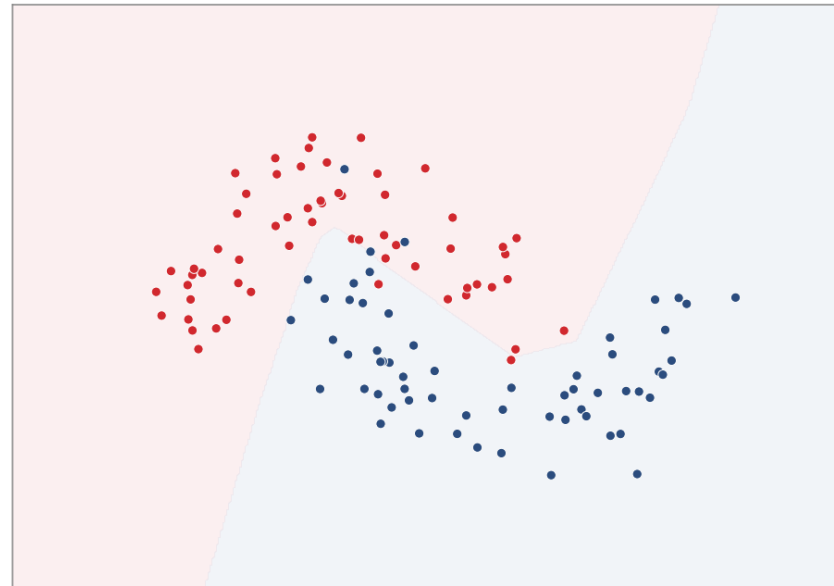
Here two classes are tangled in interleaving crescents. Logistic regression can only draw one straight line, so it cannot separate them. A small neural network bends its boundary around the shape.

When one line cannot separate the classes, depth earns its keep

Logistic regression: one straight line (88%)



Neural network: a curved boundary (95%)

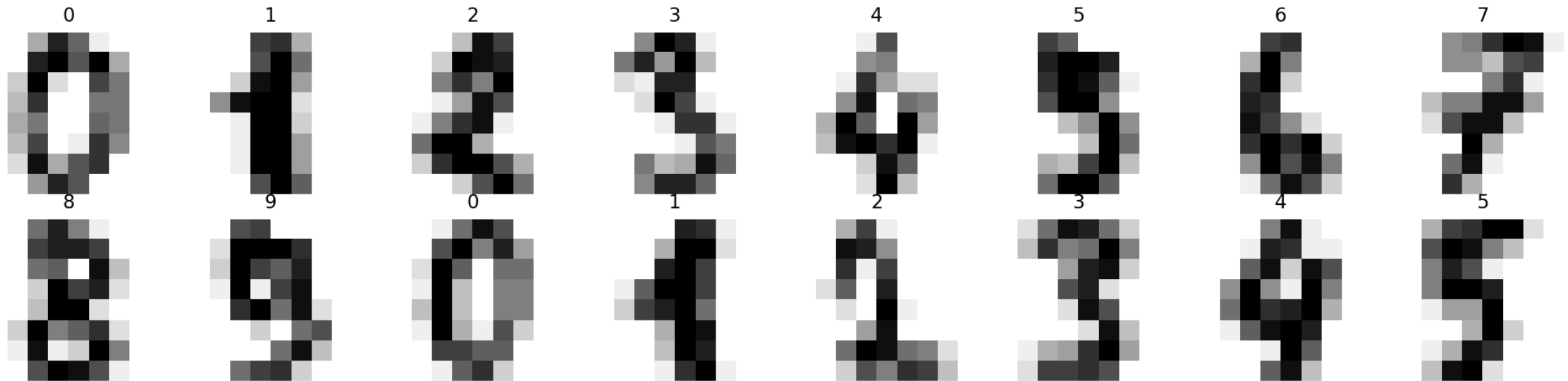


Logistic regression (one straight line, 88%) versus a neural network (a curved boundary, 95%).

THE TASK: READ HANDWRITING

The classic problem that made neural networks famous: read a handwritten digit. Each image is a tiny 8 by 8 grid, 64 pixel values, and the model must pick 0 through 9.

Handwritten digits: 8x8 pixels, 10 classes



Humans read these instantly. The question is how much structure a model needs to match that.

BUILD A NETWORK, CHECK THE BASELINE

Scale the pixels, set a logistic baseline, then build the network and compare on the same test set.

```
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import LogisticRegression

X_scaled = StandardScaler().fit_transform(X) # neural nets train better scaled

base = LogisticRegression(max_iter=1000).fit(X_tr, y_tr) # always set a baseline

mlp = MLPClassifier(hidden_layer_sizes=(64,), activation='relu',
                    max_iter=500, random_state=42).fit(X_tr, y_tr)

print('logistic:', round(base.score(X_te, y_te), 3)) # ~0.97
print('network :', round(mlp.score(X_te, y_te), 3)) # ~0.98
```

The network edges the baseline by about a point. A small win that has to earn its cost.

ALWAYS BEAT THE BASELINE FIRST

The network won, but only by about a point. That thin margin is the lesson, not a disappointment: depth helped a little here, and it has to justify its cost.

What this tells you

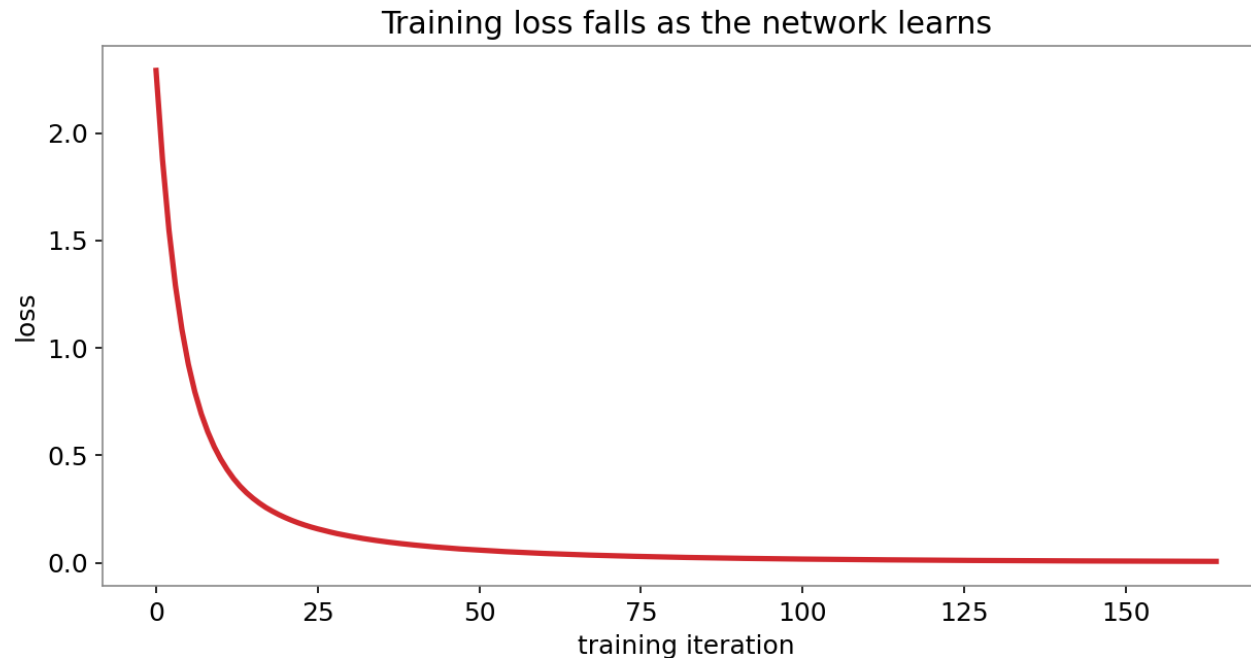
- A strong baseline means the extra complexity may not be worth it
- Neural nets earn their keep on large, complex data: images, audio, language
- For small, tidy tables, simpler models are often the smart choice

Key point:

Never accept a complex model just because it is complex. Make it beat the simple one, then weigh the gain against the cost.

WATCH IT LEARN

Unlike most models, a neural network trains iteratively: it passes over the data many times, nudging its weights each pass. The loss curve is that learning, made visible.



Loss falls steeply, then flattens. Still falling at the end means train longer.

DEPTH, OVERFITTING, AND EARLY STOPPING

More layers can learn more, but a big network will eventually memorize the training data: train accuracy near 100%, test accuracy stalling. There is a clean fix.

The fix

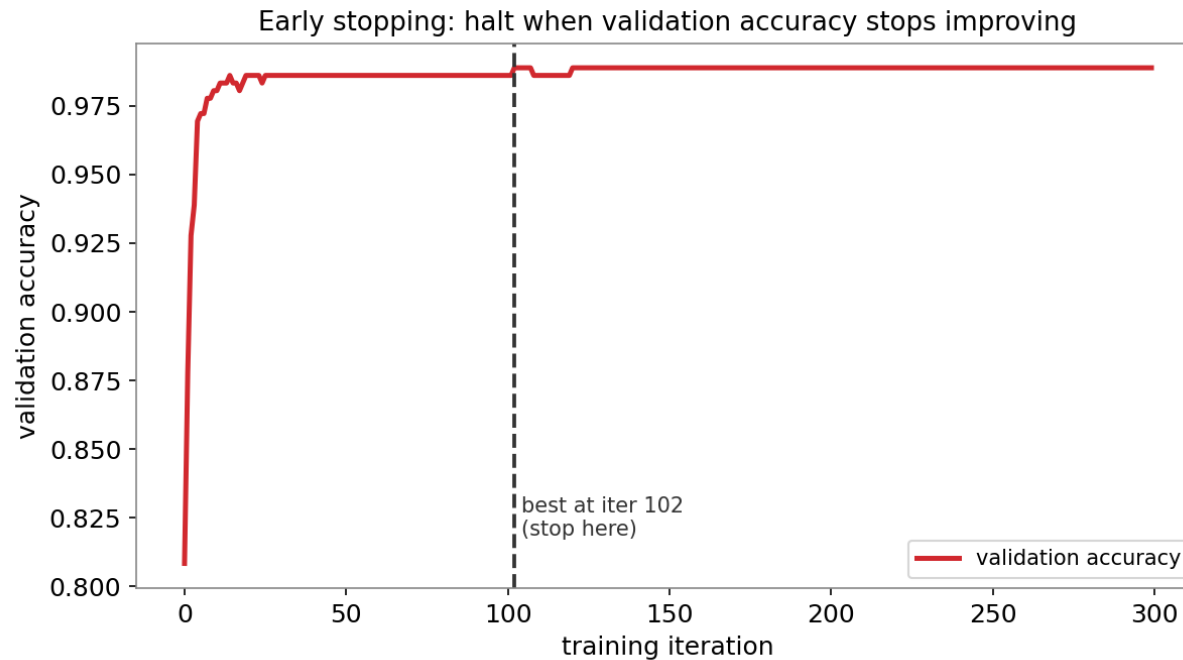
- Early stopping watches a validation set during training
- It stops when validation performance stops improving
- It is the neural-net version of limiting a tree's max_depth

Full circle:

Same lesson as the very first section: the test score is the only one that matters for deployment.

EARLY STOPPING IN ACTION

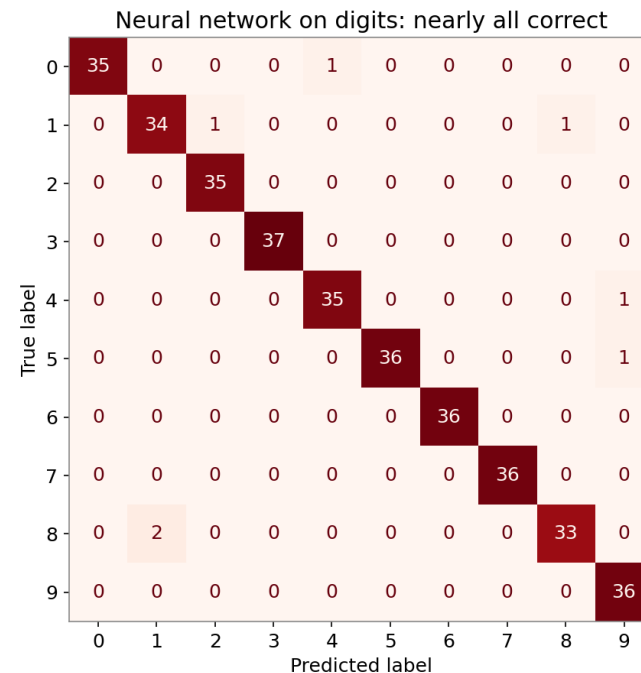
Early stopping watches accuracy on held-out data while training. It climbs, plateaus, and would eventually slip as the network begins to memorize. You keep the model from its best moment instead of its last.



Validation accuracy peaks, then flattens: stop training there, before it overfits.

WHICH DIGITS CONFUSE IT

Accuracy is one number; the confusion matrix shows exactly which digits the model mixes up. On ten classes, that is far more useful.



Just seven misses out of 360. The few errors fall on look-alikes like 8 and 1, or 4 and 9.

WHERE NEURAL NETS WIN (AND DON'T)

Neural networks are not always the answer. They earn their cost on large, complex, unstructured data and lose to simpler models on small, tidy tables.

Choose well

- They win big on images, audio, and language at scale
- They need lots of data and compute to shine
- On small structured data, trees and forests often win

Takeaway:

Match the model to the problem. Complexity is a cost, not a badge.

POP QUIZ: NEURAL NETWORKS

On a new project, your neural network scores 100% on training but 92% on test, while a plain logistic model gets 96%. What do you conclude, and what do you do?



3 MINUTES

POP QUIZ: NEURAL NETWORKS

On a new project, your neural network scores 100% on training but 92% on test, while a plain logistic model gets 96%. What do you conclude, and what do you do?

Answer:

The network overfit and did not beat the baseline. Add early stopping or shrink it, and prefer the simpler model unless depth clearly wins.

- Train far above test is the overfitting signature
- Early stopping is the neural-net equivalent of max_depth
- A model must beat the baseline to justify its complexity



KEY TAKEAWAYS

Four tools, one set of habits: read your model, control overfitting, measure the metric that matters, and make complexity earn its place.

- Decision trees give readable rules; max_depth controls overfitting
- Ensembles combine trees: forests vote, boosting corrects
- On imbalanced data, judge on recall, and fix with weights or SMOTE
- Clustering finds structure without labels; judge it with silhouette
- Neural nets add depth and risk; early stopping and a baseline keep you honest

Coming up:

Day 3 goes further: retrieval-augmented generation, agents and tools, and putting these ideas into production.

Great Work

You added four models to your toolbox. Day 3 puts them to work.



"Develop a passion for learning."